Buchstaben-Lernen-App

Technical Foundation Chapter — Master's Thesis Reference

Source-of-truth documentation generated from the Swift codebase.

Buchstaben-Lernen-App — Technical Foundation

A digital handwriting learning app for Austrian first-graders. Master’s thesis technical documentation. All claims here are verifiable against the source tree at the commit this file was last updated against. The app’s UI is German; this document is in English to match the thesis.

---

1. App overview

1.1 Purpose

The app teaches a German-speaking child how to write the Latin alphabet on an iPad. It implements a four-step pedagogical flow per letter (observe → direct → guided → freeWrite) and accompanies the child's practice with real-time audio, haptic feedback, on-device CoreML letter recognition, and per-session writing-quality assessment along four Schreibmotorik motor dimensions (form, tempo, pressure, rhythm).

The problem being addressed: the Austrian Volksschule curriculum frames handwriting acquisition as a motor-skill task that benefits from massed-then-spaced practice with explicit directionality teaching. Existing tablet handwriting apps either gate every interaction behind a "correct" tracing path (no transfer to paper) or score nothing at all (no feedback loop for parents and teachers). This app combines:

- **Pedagogical structure** (gradual release of responsibility) so the scaffolding fades as the child progresses, rather than the child learning to follow on-screen rails forever.
- **Multidimensional motor assessment** so research can analyse handwriting development across four independent dimensions instead of collapsing everything into a single accuracy number.
- **A/B-testable thesis infrastructure** so adaptive vs. non-adaptive conditions can be compared with stable cohort assignment.

1.2 Target audience

5–6 year-old children in **Volksschule 1. Klasse**. The child is the primary user. A parent / teacher accesses settings and the research dashboard via a 2-second long-press on the gear icon — children can't reach those screens accidentally.

1.3 Platform

- **iPad** in landscape, iOS 18+. Built against Swift 6.3 / Xcode 26.4.
- Both **finger** and **Apple Pencil** input. Pencil pressure + azimuth feed the pressure-control dimension; finger input is treated as zero-force.
- The package targets `iOS 26.0` and `macOS 15.0` (`Package.swift:6-9`); the BuchstabenApp Xcode wrapper hosts it on iOS.

1.4 Language

The UI is **German throughout**, including all child-facing strings, parent dashboard labels, settings, accessibility labels, and TTS utterances. The only English strings in the entire codebase are inside `#if DEBUG` blocks of the calibration tooling (and even those were translated in the latest pass — see `StrokeCalibrationOverlay.swift`).

1.5 Thesis context

The app ships with built-in A/B test infrastructure (`Core/ThesisCondition.swift`). When a participant install opts into the study via `Forschung → Studienteilnahme`, a stable UUID is deterministically mapped to one of three conditions:

- `.threePhase` — full four-phase flow (observe → direct → guided → freeWrite)
- `.guidedOnly` — guided phase only (skips scaffolding + free writing)
- `.control` — guided phase with **fixed** difficulty (no adaptive radius)

Every persisted phase-session record is tagged with the active condition (`PhaseSessionRecord.condition`), so the post-hoc CSV / TSV / JSON export distinguishes data from each arm without any extra bookkeeping. Non-enrolled installs are pinned to `.threePhase` so a casual user is never silently dropped into a degraded condition.

---

2. Architecture

2.1 File tree

`App/`

File	Lines	Role
`BuchstabenNativeApp.swift`	24	App entry. Creates the shared TracingViewModel and wires `scenePhase` → `appDidBecomeActive` / `appDidEnterBackground`.
`ContentView.swift`	382	Pre-redesign root, marked `@available(*, deprecated)`. Retained for chrome reference; not instantiated.

`Core/` (data + algorithms; no SwiftUI)

File	Lines	Role
`Models.swift`	69	`LetterAsset`, `LetterStrokes`, `StrokeDefinition`, `Checkpoint`, `TracingPoint`.
`AppWorld.swift`	49	Three-world enum (`schule`, `werkstatt`, `fortschritte`).
`LearningPhase.swift`	81	Four-phase enum: `observe`, `direct`, `guided`, `freeWrite`. German display names.
`LearningPhaseController.swift`	175	Pure-value-type FSM. Star thresholds per phase; ThesisCondition-aware `activePhases`.
`LetterOrderingStrategy.swift`	33	`motorSimilarity`, `wordBuilding`, `alphabetical` — explicit ordering tables.
`SchriftArt.swift`	63	Five script enum cases. Currently bundled: Druckschrift (Primae) + Schreibschrift (Playwrite AT).
`WritingMode.swift`	small	`guided` vs `freeform`.
`ThesisCondition.swift`	97	A/B condition + ParticipantStore with stable UUID assignment.
`OverlayQueueManager.swift`	132	Serialised FIFO of `CanvasOverlay` cases. Modal overlays carry nil duration (wait for explicit dismiss).
`AudioControlling.swift`	small	Protocol seam for the audio engine.
`PlaybackStateMachine.swift`	small	Pure FSM idle ↔ active.
`AudioEngine.swift`	382	⚠ **STABLE & FRAGILE** — do NOT modify. AVAudioEngine + AVAudioUnitTimePitch for time-stretching playback. Lifecycle observers via `nonisolated(unsafe)` static dict.
`SpeechSynthesizer.swift`	162	German `AVSpeechSynthesizer` wrapper. `ChildSpeechLibrary` centralises every TTS phrase.

File	Lines	Role
`StrokeTracker.swift`	96	Checkpoint proximity tracker. Owns `progress[]` + `radiusMultiplier`.
`HapticEngine.swift`	156	Protocol + `CoreHapticsEngine` + `UIKitHapticEngine` fallback + `NullHapticEngine` for tests.
`DifficultyAdaptation.swift`	127	`MovingAverageAdaptationPolicy` with hysteresis + `FixedAdaptationPolicy` for control arm.
`ProgressStore.swift`	282	Per-letter progress JSON store. Carries phaseScores, speedTrend (cap 5), recognitionAccuracy + recognitionSamples (cap 10), paperTransferScore, lastVariantUsed, freeformCompletionCount.
`ParentDashboardStore.swift`	372	Session + phase-record JSON store. PhaseSessionRecord stores all four Schreibmotorik dimensions. accuracySamples cap 200.
`ParentDashboardExporter.swift`	232	CSV / TSV / JSON exporter. 13-column per-phase rows.
`StreakStore.swift`	182	Daily-streak JSON store + RewardEvent enum.
`LocalNotificationScheduler.swift`	small	Daily reminder with quiet hours + streak-aware copy.
`OnboardingCoordinator.swift`	165	7-step state machine.
`CloudSyncService.swift`	small	Protocol + NullSyncService + SyncCoordinator (CloudKit-ready, not live).
`LetterScheduler.swift`	152	Spaced-repetition prioritiser. Ebbinghaus-style decay.
`LetterStars.swift`	small	phaseScores → 0…4 star count.
`LetterRecognizer.swift`	354	`CoreMLLetterRecognizer` + `StubLetterRecognizer`. nonisolated, runs on `Task.detached`.
`ConfidenceCalibrator.swift`	119	Confusable-pair penalty + history boost shim.
`FreeWriteScorer.swift`	372	Discrete Fréchet, four-dimension `WritingAssessment`, Hausdorff form-accuracy.
`FreeformWordList.swift`	small	Demo word library for freeform-word mode.
`GlyphStrokeExtractor.swift`	small	CoreText path → checkpoint extraction (debug calibrator).
`PrimaeLetterRenderer.swift`	small	OTF-based glyph render → UIImage. SchriftArt-aware.
`PBMLoader.swift`	small	PBM bitmap fallback.
`LetterAnimationGuide.swift`	small	Per-stroke animation timing model.
	small	Per-letter user-calibrated stroke persistence.

File	Lines	Role
`CalibrationStore.swift`		

`Features/Tracing/`

File	Lines	Role
`TracingViewModel.swift`	2052	`@MainActor` `@Observable` central coordinator. Touch dispatch, phase advance, scoring, recognition orchestration.
`TracingDependencies.swift`	small	DI bundle. `.live` defaults; `.stub` swaps for tests.
`TracingCanvasView.swift`	697	SwiftUI Canvas: glyph + ghost + ink + KP overlay. Pencil + finger UIKit overlays.
`FreeformController.swift`	113	Owns freeform state (mode, sub-mode, target word, drawing buffers, recognition state, debounce task).
`FreeWritePhaseRecorder.swift`	153	Owns the four freeWrite buffers + session timing + Schreibmotorik scoring.
`FreeformWritingView.swift`	~720	Blank-canvas writing UI. Verbal-only result popup (no metrics shown to children).
`RecognitionFeedbackView.swift`	140	German verbal-only badge for freeWrite recognition.
`CompletionCelebrationOverlay.swift`	small	Stars + “Weiter” button.
`PaperTransferView.swift`	95	3 s reference → 10 s write-on-paper → 3-emoji self-assessment. Speaks each prompt.
`PhaseDotIndicator.swift` / `PhaseIndicatorView.swift`	small	Phase progress visuals.
`LetterPickerBar.swift` / `LetterWheelPicker.swift` / `SequencePickerBar.swift`	small	Letter / word selection chrome.
`DebugAudioPanel.swift`	small	DEBUG-only audio tuning sliders.
`StrokeCalibrationOverlay.swift`	521	DEBUG calibration UI with per-script persistence.
`PlaybackController.swift` / `AnimationGuideController.swift` / `TransientMessagePresenter.swift`	small	Per-VM controllers built via factories.
`InputModeDetector.swift` / `InputPreset.swift`	small	Finger / pencil hysteresis.
	small	Word-mode grid.

File	Lines	Role
`LetterCell.swift` / `SequenceGridControl-ler.swift` / `GridLayoutCalculator.swift` / `TracingSequence.swift`		

`Features/Library/`

File	Lines	Role
`LetterRepository.swift`	545	Bundle scan + cache fallback + variant loader.
`LetterCache.swift`	small	JSONLetterCache disk cache.

`Features/Navigation/`

File	Lines	Role
`MainAppView.swift`	86	Root host. `@AppStorage` persists active world.
`WorldSwitcherRail.swift`	162	64 pt rail with world buttons + 2 s long-press gear.
`WorldPalette.swift`	small	Per-world palette.

`Features/Worlds/`

File	Lines	Role
`SchuleWorldView.swift`	~280	World 1. Hosts `TracingCanvasView` + queue-driven overlays.
`WerkstattWorldView.swift`	126	World 2. Mode-card panel + `FreeformWritingView`.
`FortschritteWorld-View.swift`	~230	World 3. Star + streak + letter gallery + fluency footer.

`Features/Onboarding/`

File	Lines	Role
`OnboardingView.swift`	569	First-run flow with one animated demo per phase.

`Features/Parent/`

File	Lines	Role
`ParentAreaV-iew.swift`	153	Parental-gate host. NavigationSplitView with overview / research / settings / export.
`ResearchDashboard-View.swift`	343	Research-only metrics view. Schreibmotorik, KI predictions vs expected, condition arms, scheduler r, raw phase records.

`Features/Dashboard/`

File	Lines	Role
`ParentDashboardView.swift`	~250	Adult-grade overview.
`PhaseRatesView.swift` / `PracticeTrendChart.swift`	small	Charts.
`SettingsView.swift`	99	Schriftart / ordering / freeform / paper-transfer / study-enrolment / restart-onboarding.

`Resources/`

- `Letters/<X>/strokes.json` — 26 uppercase + 26 lowercase + Ä Ö Ü ß. Demo set (A F I K L M O) plus full alphabet placeholders for completion.
- `Letters/<X>/strokes_variant.json` — alternate stroke order (uppercase F, H, lowercase r).
- `Letters/<X>/strokes_schulschrift.json` — Schreibschrift script strokes for the demo letters.
- `Letters/<X>/<X>.pbm` + `<X><n>.mp3` — bitmap glyph fallback + recorded audio variants.
- `Fonts/Primae-*.otf` (12 weights) + `PlaywriteAT-Regular.ttf` + `PlaywriteAT-OFL.txt` (license).
- `ML/GermanLetterRecognizer.mlpackage/` — CoreML model + manifest.
- `letter_set.json` — `active_letters: ["A","F","I","K","L","M","O"]`.

`BuchstabenNativeTests/`

40+ test files using **Swift Testing** (`@Test`, `@Suite`, `#expect`). Notable: `WritingAssessmentTests`, `LetterSchedulerTests`, `ParentDashboardStoreTests`, `ParentDashboardExporterTests`, `ProgressStoreTests`, `LearningPhaseControllerTests`, `StrokeTrackerTests`, `ConfidenceCalibratorTests`, `TracingViewModelTests`, `EndToEndTracingSessionTests`. Stubs in `TestFixtures.swift`.

`.github/workflows/`

- `ios-build.yml` — main GitHub Actions runner, macos-26 / Xcode 26.4. Runs `xcodebuild test` on iPad simulator.
- `ipad-device-test.yml` — self-hosted MacBook physical-device test job.
- `legacy-build.yml` — historical build job retained for reference.

2.2 Dependency graph

```
BuchstabenNativeApp
  └── @State vm = TracingViewModel()       ← single shared instance
       └── TracingDependencies.live (injected)
            ├── AudioEngine          (AudioControlling)
            ├── JSONProgressStore    (ProgressStoring)
            ├── CoreHapticsEngine    (HapticEngineProviding)
            ├── MovingAverageAdaptationPolicy / FixedAdaptationPolicy
            ├── LetterRepository → JSONLetterCache
            ├── JSONStreakStore
            ├── JSONParentDashboardStore
            ├── JSONOnboardingStore
            ├── LocalNotificationScheduler
            ├── SyncCoordinator → NullSyncService
            ├── ThesisCondition.defaultForInstall
            ├── SchriftArt        (UserDefaults-persisted)
            ├── LetterOrderingStrategy   (UserDefaults-persisted)
            ├── enablePaperTransfer / enableFreeformMode (UserDefaults)
            ├── CoreMLLetterRecognizer  (LetterRecognizerProtocol)
            └── AVSpeechSpeechSynthesizer (SpeechSynthesizing)

TracingViewModel
  ├── private freeWriteRecorder = FreeWritePhaseRecorder()
  ├── private freeform = FreeformController()
  ├── private grid = SequenceGridController(...)
  ├── private detector = InputModeDetector()
  ├── private phaseController: LearningPhaseController(condition:)
  ├── private adaptationPolicy: any AdaptationPolicy
  ├── private letterScheduler: LetterScheduler
  ├── private calibrationStore: CalibrationStore
  └── let overlayQueue = OverlayQueueManager()
```

`TracingViewModel` is `@MainActor @Observable`. SwiftUI views read its properties via `@Environment(TracingViewModel.self)`. The VM forwards ~65 observable properties (some via direct storage, some via computed forwarders to the FreeWritePhaseRecorder / FreeformController / phaseController / animation / messages collaborators).

There are **no circular dependencies**. The Core layer never imports from Features. The dependency graph is acyclic (verified by `Package.swift` being a single target with no internal sub-modules).

2.3 Design patterns

- **MVVM**: SwiftUI views are passive observers; `TracingViewModel` is the single source of truth. Views never read from stores directly — they always go through the VM.
- **Protocol-oriented + dependency injection**: every external collaborator (audio, progress store, haptic engine, letter recognizer, speech synth, dashboard store, onboarding store, streak store, sync service, notification center, adaptation policy) is protocol-typed in `TracingDependencies`. Production uses live implementations; tests inject `Stub*` types from `TestFixtures.swift`.
- **Pure-value-type FSMs** for the deterministic logic: `LearningPhase`, `LearningPhaseController`, `PlaybackStateMachine`. They have no I/O and are exhaustively unit-tested without mocks.

- **Composition over inheritance**: the W8 God-object refactor pulled `FreeWritePhaseRecorder` and `FreeformController` out of the VM and composed them in. The VM forwards observable properties so views notice nothing.
- **Factories for per-VM controllers**: `TracingDependencies` holds factory closures (`makePlaybackController`, `makeMessagePresenter`, `makeAnimationGuide`, `makeCalibrationStore`, `makeLetterScheduler`) so tests can swap in instant-sleeper variants without subclassing.

2.4 Data flow

A single touch on the canvas:

```
User finger / pencil
  ↓ (UIKit gesture recognizer in TracingCanvasView)
TracingViewModel.beginTouch / updateTouch / endTouch
  ↓
StrokeTracker.update(normalizedPoint:)        ← checkpoint hit detection
  ↓ (callback)
HapticEngine.fire(.checkpointHit)
  ↓
PlaybackController.request(.active)            ← gated on velocity, feedbackIntensity
  ↓
AudioEngine.setAdaptivePlayback(speed:, hBias:)

(simultaneously, during freeWrite)
  ↓
FreeWritePhaseRecorder.record(point:, t:, force:, canvasSize:)
  ↓ (later, on phase advance)
FreeWriteScorer.score(...) → WritingAssessment
  ↓
ParentDashboardStore.recordPhaseSession(... , assessment:)
  ↓ (background Task)
JSON write to Application Support (atomic, debounced)
```

A phase advance:

```
TracingViewModel.advanceLearningPhase
  ↓
phaseController.advance(score:)              ← mutates currentPhase
  ↓
overlayQueue.enqueue(.kpOverlay)              ← if wasInFreeWrite
overlayQueue.enqueue(.paperTransfer(...))     ← if enablePaperTransfer
recordPhaseSessionCompletion()
  └── overlayQueue.enqueue(.celebration(stars:))
  └── speech.speak(praise(starsEarned:))
  └── ParentDashboardStore.recordPhaseSession (per phase)
  └── ProgressStore.recordCompletion (per letter in word)
  └── StreakStore.recordSession
  └── adaptationPolicy.record(AdaptationSample)
  └── strokeTracker.radiusMultiplier = currentDifficultyTier.radiusMultiplier

(in parallel, async)
runRecognizerForFreeWrite → CoreMLLetterRecognizer.recognize (Task.detached)
  ↓
overlayQueue.enqueueBeforeCelebration(.recognitionBadge(r))
speech.speak(recognition phrase)
ProgressStore.recordRecognitionSample
```

3. Learning pipeline — detailed

3.1 Phase: Observe (Anschauen)

What the child sees and does - The letter glyph rendered in the active script (Druckschrift or Schreibschrift). - An animated guide dot tracing each stroke in canonical order. - Numbered start dots indicating where each stroke begins. - A “Schau mal genau hin.” prompt is spoken via TTS on phase entry. - The child taps anywhere on the overlay to advance — or the phase auto-advances after the second full animation cycle so a child who can’t read isn’t stuck.

What the code does - `LearningPhaseController.currentPhase = .observe` (initial state). - `phaseController.isTouchEnabled` returns `false` — the canvas does not record touches. - `TracingViewModel.startGuideAnimation()` calls `AnimationGuideController.start(strokes:)`. The controller publishes `guidePoint` which `TracingCanvasView` renders as an orange dot. - Auto-advance is implemented in `TracingViewModel.startGuideAnimation` (line 1058):

```
swift animation.onCycleComplete = { [weak self] in guard let self else { return } self.observeCycleCount += 1 if self.observeCycleCount >= 2, self.phaseController.currentPhase == .observe { self.completeObservePhase() } }
```

Recorded data - `phaseScores[.observe] = 1.0` on completion (pass/fail). - A `PhaseSessionRecord` row with `phase: "observe"`, `completed: true`, `score: 1.0`.

Trigger to next phase - `completeObservePhase()` → `advanceLearningPhase()` → phase becomes `.direct`. The verbal prompt for `.direct` (“Tippe die Punkte der Reihe nach.”) is spoken immediately.

3.2 Phase: Direct (Richtung lernen)

What the child sees and does - The same letter glyph is shown. - A single numbered dot pulses at the next stroke’s start position. - The child taps the dot. On a correct tap: confirmation haptic, brief directional arrow along the stroke path, and (on the very first tap) a replay of the letter-name audio. On a wrong tap: gentle haptic, the correct dot pulses. - After all strokes’ start dots have been tapped in order, the phase auto-advances.

What the code does - `LearningPhase.direct.rawValue == 1` — placed between `observe` (0) and `guided` (2). - `DirectPhaseDotsOverlay` (in `TracingCanvasView.swift`) renders only the **next-expected** dot so visually overlapping starts (A’s two diagonals share the apex) don’t stack. - `TracingViewModel.tapDirectDot(index:)` (line 1240):

```
swift if index == directNextExpectedDotIndex { directTappedDots.insert(index) haptics.fire(.checkpointHit) // play letter audio on first tap; show directional arrow directArrowStrokeIndex = index // defer phase advance until arrow finishes (1.2 s) } else { haptics.fire(.offPath) directPulsingDot = true // resets after 700 ms }
```

- Empty-stroke letters skip direct: `load(letter:)` calls `phaseController.advance(score: 1.0)` twice in succession to land in `.guided`.

Recorded data - `phaseScores[.direct] = 1.0` (pass/fail). - A `PhaseSessionRecord` row with `phase: "direct"`.

Trigger to next phase - Last correct tap → 1.2-second arrow window → `advanceLearningPhase()`.

3.3 Phase: Guided (Nachspuren)

What the child sees and does - The letter glyph plus a faint blue ghost line over each stroke. - Real-time green ink follows the finger / pencil. - A "Jetzt fährst du die Linien nach." prompt is spoken on phase entry. - Audio plays in real time when the touch is on-path, with playback speed mapping to writing velocity (slow tracing → slow audio, fast → fast). - Haptic ticks on every checkpoint hit.

What the code does - `phaseController.useCheckpointGating == true`: `StrokeTracker` enforces stroke order. Each stroke must be traced left-to-right (or the canonical direction encoded in `strokes.json`); jumping to the next stroke before completing the current one yields no progress. - `TracingViewModel.updateTouch` (line 818) feeds normalized touch points into `strokeTracker.update(normalizedPoint:)`. - Audio is gated by `feedbackIntensity > 0.3` (line 937) and `smoothedVelocity >= playbackActivationVelocityThreshold` (default 22 pt/s). - The freeWriteRecorder also tracks `strokesPerSecond` during this phase via `freeWriteRecorder.updateSpeed(completedCheckpoints:)`.

Recorded data - `phaseScores[.guided] = strokeTracker.overallProgress` (0–1) at the moment the phase advances. - `lastGuidedScore` is captured (FreeformController) so the SchuleWorld can show a verbal "Nachspuren fertig" feedback band during the freeWrite transition. - A `PhaseSessionRecord` row.

Trigger to next phase - `strokeTracker.isComplete && hasStrokes` → `advanceLearningPhase()`.

3.4 Phase: FreeWrite (Selbst schreiben)

What the child sees and does - The letter glyph remains visible but no ghost lines and no checkpoints are shown. - The child writes the letter from memory. - A "Jetzt schreibst du den Buchstaben ganz alleine." prompt is spoken on entry. - No haptics, no real-time audio (`feedbackIntensity == 0.0`). - On lift-off, the child lifts the pen and the phase ends.

What the code does - `feedbackIntensity == 0.0` so `updateTouch` skips haptics and audio (lines 891, 918, 937). - Every touch is appended to four parallel buffers in `FreeWritePhaseRecorder`: `points`, `timestamps`, `forces`, `path` (the canvas-normalised KP path). - On `advanceLearningPhase`: `swift case .freeWrite: guard let def = strokeTracker.definition else { ... } score = freeWriteRecorder.assess( reference: def, canvasSize: canvasSize ).overallScore` - `FreeWriteScorer.score` returns a `WritingAssessment` — see §4.6. - Recognition runs on a background `Task.detached`; results land asynchronously and enqueue `.recognitionBadge` ahead of the celebration via `enqueueBeforeCelebration`.

Recorded data - `phaseScores[.freeWrite] = assessment.overallScore`. - `WritingAssessment` is stored on `PhaseSessionRecord`'s four optional dimension fields. - `progressStore.recordCompletion(... recognitionResult:)` if recognition finished by then; otherwise `recordRecognitionSample` populates history when the result lands.

Trigger to session completion - `phaseController.advance(...)` returns false — last phase. The celebration overlay shows; “Weiter” loads the next recommended letter via the LetterScheduler.

---

4. Scientific methods — with citations

Each method is backed by named papers and the implementation site is explicitly identified. Where the code differs from the paper, the delta is documented.

4.1 Gradual Release of Responsibility (GRR)

Description. A pedagogical model where teacher control fades and learner responsibility rises across stages: “I do” → “We do” → “You do.” The original framing has three phases; the modern reading (Fisher & Frey, 2013) inserts a guided-practice phase between modelled demonstration and independent application.

Papers. * Pearson, P. D., & Gallagher, M. C. (1983). The instruction of reading comprehension. Contemporary Educational Psychology, 8(3), 317–344. * Fisher, D., & Frey, N. (2013). Better learning through structured teaching: A framework for the gradual release of responsibility (2nd ed.). ASCD.

Implementation. * `Core/LearningPhase.swift` — four `case`s in raw-value order `observe / direct / guided / freeWrite`. Each carries a German display name, an icon, a stable `rawName` for serialisation. * `Core/LearningPhaseController.swift` — pure value-type FSM. Field comments name the GRR mapping: observe = “I do”, direct = “Start”, guided = “We do”, freeWrite = “You do.”

How the implementation differs. The classic GRR has three phases; this app inserts a fourth (`direct`) between observe and guided to explicitly teach stroke directionality (see §4.4). All four phases participate in gradual release: scaffolding visible (observe + direct) → scaffolding interactive (guided) → scaffolding withdrawn (freeWrite).

4.2 Guidance Hypothesis — Fading Feedback

Description. Schmidt & Lee’s guidance hypothesis predicts that augmented feedback during practice is helpful for performance but can become a crutch — long-term retention improves when feedback fades as the learner internalises the task. The app implements an explicit phase-dependent feedback intensity gate.

Paper. Schmidt, R. A., & Lee, T. D. (2005). Motor control and learning: A behavioral emphasis (4th ed.). Human Kinetics.

Implementation. * `Features/Tracing/TracingViewModel.swift` — `feedbackIntensity` computed property (line 204):

`swift var feedbackIntensity: CGFloat { switch phaseController.currentPhase { case .observe: return 1.0 case .direct: return 1.0 case .guided: return 0.6 case .freeWrite: return 0.0 } }`

- The gate is read at three call sites in `updateTouch`:
- line 891: `if !wasComplete && isNowComplete, feedbackIntensity > 0 { haptics.fire(...) }`
- line 918: `if (...stroke or checkpoint changed...), feedbackIntensity > 0 { haptics.fire(...) }`

- line 937: `let shouldBeActive = ... && feedbackIntensity > 0.3`

→ audio cuts off entirely below 0.3 (so freeWrite is silent), haptics cut off entirely below `> 0`.

How the implementation differs. The paper describes a continuous fade; this implementation uses four discrete phase-pinned levels so the gating thresholds are auditable and can be tested. The 0.3 threshold for audio is chosen so the `.guided` phase (0.6) keeps audio on while `.freeWrite` (0.0) silences it.

4.3 Knowledge of Performance (KP) Visual Overlay

Description. Danna & Velay distinguish Knowledge of Results (KR — "your score was X") from Knowledge of Performance (KP — "you deviated here"). Children under 8 benefit more from KP than KR because the process feedback is directly actionable.

Paper. Danna, J., & Velay, J.-L. (2015). Basic and supplementary sensory feedback in handwriting. Frontiers in Psychology, 6, 169.

Implementation. * `Features/Tracing/TracingCanvasView.swift` — `freeWriteKPOverlay()` function (line 224). After freeWrite completes, the overlay superimposes: - the reference strokes in **blue** (line width 8, opacity 0.4) - the child's path in **green** (line width 4) on a darkened background. The child sees exactly where their trace diverged from the reference.

- The overlay is now driven by the OverlayQueueManager: rendered when `vm.overlayQueue.currentOverlay == .kpOverlay` (line 346 of TracingCanvasView). Auto-dismisses after 3 s (the queue's default duration for `.kpOverlay`) or on tap (`vm.overlayQueue.dismiss()`).

How the implementation differs. Paper compares KP vs KR empirically; this app provides KP as the default visualisation after freeWrite. KR (the numeric score) is reserved for the parent-gated research dashboard and never shown to the child.

4.4 Directionality Teaching

Description. Thibon, Gerber & Kandel show that children under 8 store motor programs for individual stroke segments rather than the letter as a whole. Explicitly teaching where each stroke starts (and in what order) supports the formation of these per-segment motor programs.

Paper. Thibon, L. S., Gerber, S., & Kandel, S. (2018). The elaboration of motor programs for the automation of letter production. Acta Psychologica, 182, 200–211.

Implementation. * `Core/LearningPhase.swift` — explicit `.direct` case between `.observe` and `.guided`. * `Core/LearningPhaseController.swift` — `useCheckpointGating: false` and `showCheckpoints: false` for direct (the dot overlay handles its own rendering). * `Features/Tracing/TracingCanvasView.swift` — `DirectPhaseDotsOverlay` inner View. Renders the next-expected stroke's start dot only. * `Features/Tracing/TracingViewModel.swift` — `tapDirectDot(index:)` drives the correct/wrong tap logic + 1.2-second arrow window before phase advance.

How the implementation differs. Paper observes the phenomenon; this app builds an explicit pedagogical phase around it. The 1.2- second arrow timing was chosen empirically so a 5-year-old can perceive the direction cue before the next dot pulses.

4.5 Motor Similarity Letter Ordering

Description. Berninger et al. argue that grouping letters by shared stroke patterns ("motor similarity") during early instruction promotes faster acquisition than alphabetical or random ordering, by letting the child re-use motor programs across letters that share stroke shapes.

Paper. Berninger, V. W., Abbott, R. D., Jones, J., Wolf, B. J., Gould, L., Anderson-Youngstrom, M., Shimada, S., & Apel, K. (2006). Early development of language by hand: Composing, reading, listening, and speaking connections; three letter-writing modes; and fast mapping in spelling. Developmental Neuropsychology, 29(1), 61–92.

Implementation. * `Core/LetterOrderingStrategy.swift` — three strategies as an enum:

```swift enum LetterOrderingStrategy: String, Codable, CaseIterable { case motorSimilarity, wordBuilding, alphabetical

```
func orderedLetters() -> [String] {
    switch self {
    case .motorSimilarity:
        return ["I","L","T","F","E","H","A","K","M","N","V","W","X","Y","Z",
                "C","O","G","Q","S","U","J","B","D","P","R"]
    case .wordBuilding:
        return ["M","A","L","E","I","O","S","R","N","T","D","U","H","G","K",
                "W","B","P","F","J","V","C","Q","X","Y","Z"]
    case .alphabetical:
        return (Unicode.Scalar("A").value...Unicode.Scalar("Z").value)
            .compactMap { Unicode.Scalar($0) }.map { String($0) }
    }
}
```

} ```

- `TracingViewModel.visibleLetterNames` (line 1321) sorts the loaded letters by the active ordering's rank map.
- `SettingsView.swift` exposes the picker.

**How the implementation differs.** Paper proposes the principle but not a specific 26-letter sequence; the motorSimilarity ordering above is the app's own choice (vertical → horizontal → diagonal → curved). The `wordBuilding` strategy is an alternative inspired by Austrian practice that lets a child form recognisable words (M, A, L → MAL) within the first dozen letters.

## 4.6 Schreibmotorik 4-Dimension Assessment

**Description.** Marquardt & Söhl (Schreibmotorik Institut) characterise handwriting motor competency along four dimensions: form accuracy, tempo (speed consistency), Druck (pressure control), and rhythm (fluency). Each is independently measurable; an aggregate score should weight them rather than collapse to one number.
```

Paper. Marquardt, C., & Söhl, K. (2016). Schreibmotorik: Schreiben lernen leicht gemacht — Eine Praxishilfe der STEP– Schreibmotorik-Initiative. Cornelsen.

Implementation. * `Core/FreeWriteScorer.swift` — `WritingAssessment` struct (line 19):

```swift struct WritingAssessment: Codable, Equatable { let formAccuracy: CGFloat // discrete Fréchet let tempoConsistency: CGFloat // 1 − CV² of inter-point intervals let pressureControl: CGFloat // 1 − variance of pencil force let rhythmScore: CGFloat // active-time / total-time

```
var overallScore: CGFloat {
    formAccuracy * 0.40 + tempoConsistency * 0.25
    + pressureControl * 0.15 + rhythmScore * 0.20
}
```

} ```

Weights sum to **1.0 exactly** (verified by `WritingAssessmentTests.testWeightedAccuracySumsToOne`).

- Each dimension's algorithm:
- `formAccuracy` (line 204): discrete Fréchet distance vs reference polyline, normalised by `reference.checkpointRadius * 3.0`.
- `tempoConsistency` (line 221): $CV^2$ of inter-point timestamp deltas; excludes gaps > 0.5 s as pen-lifts.
- `pressureControl` (line 244): 1 − normalised variance over non-zero pencil forces; finger touches (force 0) score 1.0 (no pressure data to fault).
- `rhythmScore` (line 261): ratio of active drawing time to total session time.
- `Core/ParentDashboardStore.swift` — `PhaseSessionRecord` carries all four dimensions as optionals (nil for non-freeWrite phases).
- `ResearchDashboardView.swift` displays the rolling averages with the weight (× 0.40, × 0.25, × 0.15, × 0.20) shown.
- CSV / TSV export (line 64): four dimensions as separate columns.

**How the implementation differs.** Paper describes the dimensions qualitatively; the formulas above are the app's choice of operational definitions. The 0.40 / 0.25 / 0.15 / 0.20 weights also represent a pedagogical priority (form first, consistency second) rather than a peer-reviewed weighting.

## 4.7 Writing Speed / Automatization Tracking

**Description.** The KMK Vereinbarung (German Conference of Cultural Ministers, 2024 update) explicitly names Automatisierung der Bewegungsabläufe (automatisation of writing movement) as a Grundschule goal. Speed-trend over sessions is the standard proxy for this.

**Paper.** Kultusministerkonferenz (2024). Vereinbarung zur Arbeit in der Grundschule. Beschluss der Kultusministerkonferenz (revised).

**Implementation.** * `Features/Tracing/FreeWritePhaseRecorder.swift` — owns `strokesPerSecond`, recomputed on each touch:
```

```swift
func updateSpeed(completedCheckpoints: Int, now: CFTimeInterval = CACurrentMediaTime()) { guard activePhaseStart > 0 else { return } let elapsed = now - activePhaseStart guard elapsed > 0.1 else { return } strokesPerSecond = CGFloat(completedCheckpoints) / CGFloat(elapsed) }
```

- `Core/ProgressStore.swift` — `LetterProgress.speedTrend: [Double]?` capped at 5 entries (line 148):

```swift
if trend.count > 5 { trend.removeFirst(trend.count - 5) }
```

- `Core/LetterScheduler.swift` — `automatizationBonus(trend:)` (line 136) reads the speed trend and returns a priority adjustment: improving speed → negative bonus (deprioritise letters being automatised), stagnant or declining → small positive boost.
- `TracingViewModel.writingSpeedTrend` (line 1466) aggregates across all letters: `improving` / `stable` / `declining` based on relative gain thresholds (±10 %).
- `FortschritteWorldView` shows this as a verbal trend pill ("aufwärts / stabil / abwärts") with no numbers.

How the implementation differs. KMK names the goal but not the metric. `strokesPerSecond` (checkpoints completed per second) is the app's choice; it normalises across letters of different stroke-count because it's a rate.

4.8 Paper Transfer Self-Assessment

Description. Alamargot & Morin show that tablet writing has lower friction than paper, which can disrupt motor-control strategies. A parental-research mode lets the child write on real paper after each freeWrite trial and self-assess the result.

Paper. Alamargot, D., & Morin, M.-F. (2015). Does handwriting on a tablet screen affect students' graphomotor execution? A comparison between Grades Two and Nine. Human Movement Science, 44, 32–41.

Implementation. * `Features/Tracing/PaperTransferView.swift` — full implementation. Three phases driven by a `Phase` enum: `.showLetter` (3 s reference letter shown) → `.writePaper` (10 s prompt to write on paper) → `.assess` (three emoji buttons 😟 / 😐 / 😊 with scores 0.0 / 0.5 / 1.0). * Each phase speaks its prompt via TTS: `paperTransferShow` / `paperTransferWrite` / `paperTransferAssess` from `ChildSpeechLibrary`. * `TracingViewModel.submitPaperTransfer(score:)` records the score on `LetterProgress.paperTransferScore` and dismisses the queue. * Gated behind the `enablePaperTransfer` Settings toggle (off by default — research-only).

How the implementation differs. Paper compares grade 2 vs grade 9; this app implements the self-assessment as an opt-in research feature. Children's self-assessment is a known noisy signal but useful for transfer-awareness analysis post-hoc.

4.9 Letter Variant Exploration

Description. The Grundschulverband recommends exposing children to multiple legitimate variants of the same letter (e.g. two stroke orderings for F, H) early, so motor programs stay flexible rather than locking to one canonical form.

Paper. Grundschulverband. (2024). Grundschrift — Förderkartei zur Schreibmotorik. Frankfurt am Main: Grundschulverband.

Implementation. * `Core/Models.swift` — `LetterAsset.variants: [String]?` lists alternate stroke-order IDs (e.g. `["variant"]`). * `Features/Library/LetterRepository.swift` — bundle scan looks for `Letters/<X>/strokes_variant.json` and registers it as a variant (line 295). * `Features/Library/LetterRepository.swift:529` — `loadVariantStrokes(for:variantID:)` loads the variant on demand. * `TracingViewModel.toggleVariant()` (line 567) swaps the active stroke set between standard and variant; the canvas re-maps checkpoints accordingly. * Variants currently shipped: uppercase **F**, uppercase **H**, lowercase **r**.

How the implementation differs. The Förderkartei lists many letters; the app ships variants for three (F, H, r) as a demo. The infrastructure scales to any letter that ships a `strokes_variant.json`.

4.10 Spaced Repetition

Description. Ebbinghaus's forgetting curve R(t) = e^(−t/S) predicts exponential memory decay. Cepeda et al.'s meta-analysis confirms that spaced practice outperforms massed practice when inter-session interval is calibrated to the desired retention.

Papers. * Ebbinghaus, H. (1885). Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie. Duncker & Humblot. * Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., & Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. Psychological Bulletin, 132(3), 354–380.

Implementation. * `Core/LetterScheduler.swift` — weighted prioritiser. Three components in [0, 1]:

```swift let recencyUrgency = 1.0 - exp(-daysSince / memoryStabilityDays) // Ebbinghaus let accuracyDeficit = 1.0 - p.bestAccuracy let novelty = 1.0 / (1.0 + Double(p.completionCount))

let priority = recencyUrgency * 100 * recencyWeight // 0.40 + accuracyDeficit * 100 * accuracyWeight // 0.35 + novelty * 100 * noveltyWeight // 0.25 + speedBonus // automatisation pull-down ```

`memoryStabilityDays = 7.0` (matches typical word-list half-decay).

- `TracingViewModel.loadRecommendedLetter()` calls `letterScheduler.recommendNext(available:, progress:)` and loads the highest-priority letter.

**How the implementation differs.** Ebbinghaus's curve has a single parameter (memory stability); this app combines that with accuracy and novelty terms because empirical handwriting practice involves more than memory decay (children also benefit from low-completion exposure). The combination is multi-objective rather than purely predictive.

## 4.11 Adaptive Difficulty (Zone of Proximal Development)

**Description.** Vygotsky's ZPD: tasks just outside what a learner can accomplish independently maximise learning. Operationally: difficulty should rise when accuracy is consistently high and fall when it drops.
```

Conceptual root. Vygotsky, L. S. (1978). Mind in society: The development of higher psychological processes. Harvard University Press. (Original work compiled posthumously from 1930s manuscripts.)

Implementation. * `Core/DifficultyAdaptation.swift` — `MovingAverageAdaptationPolicy`:

```swift
init(windowSize: Int = 10, hysteresisCount: Int = 3, promotionAccuracyThreshold: CGFloat = 0.85, demotionAccuracyThreshold: CGFloat = 0.55, initialTier: DifficultyTier = .standard)
```

- 10-sample moving accuracy window
- Promote when avg ≥ 0.85 for 3 consecutive evaluations
- Demote when avg ≤ 0.55 for 3 consecutive evaluations
- Three tiers: `.easy` (radius × 1.5), `.standard` (× 1.0), `.strict` (× 0.65)
- The radius multiplier is applied to `StrokeTracker.radiusMultiplier` whenever a letter session completes (`commitCompletion`, line 1451).
- `FixedAdaptationPolicy(currentTier: .standard)` is used for the `.control` thesis condition so adaptation is disabled and a clean comparison is possible.

How the implementation differs. ZPD is a qualitative concept; the operationalisation as a moving-average + hysteresis ladder is the app's choice. The thresholds (0.85 / 0.55) and the 3-evaluation hysteresis were chosen to prevent ping-ponging.

4.12 Fréchet Distance Scoring

Description. The discrete Fréchet distance is the maximum of the minimum point-to-point distances along two parameterised polylines — intuitively, "how tightly can a leash bound a person walking along curve A and a dog walking along curve B?" Formally it is the metric distance on the space of curves.

Papers. * Fréchet, M. (1906). Sur quelques points du calcul fonctionnel. Rendiconti del Circolo Matematico di Palermo, 22(1), 1–72. * Eiter, T., & Mannila, H. (1994). Computing discrete Fréchet distance. Technical Report CD-TR 94/64, Christian Doppler Lab for Expert Systems, TU Vienna. * Alt, H., & Godau, M. (1995). Computing the Fréchet distance between two polygonal curves. International Journal of Computational Geometry & Applications, 5(1n02), 75–91.

Implementation. * `Core/FreeWriteScorer.swift:283` — discrete Fréchet via O(nm) dynamic programming on a flat array (cache-line efficient, iterative bottom-up to avoid stack overflow):

```swift
static func discreteFrechetDistance(_ p: [CGPoint], _ q: [CGPoint]) -> CGFloat { let n = p.count, m = q.count var dp = [CGFloat](repeating: 0, count: n * m) for i in 0..<n { for j in 0..<m { let d = dist(p[i], q[j]) let idx = i * m + j if i == 0 && j == 0 { dp[idx] = d } else if i == 0 { dp[idx] = max(d, dp[j - 1]) } else if j == 0 { dp[idx] = max(d, dp[(i - 1) * m]) } else { let prev = min(dp[(i - 1) * m + j], min(dp[i * m + (j - 1)], dp[(i - 1) * m + (j - 1)])) dp[idx] = max(d, prev) } } } return dp[n * m - 1] }
```

- `formAccuracy(tracedPoints:reference:)` resamples both polylines to the same point count, runs Fréchet, and normalises by `reference.checkpointRadius * 3.0`.

- `formAccuracyShape(tracedPoints:reference:)` is an order-independent variant for free-form mode: it densifies each reference stroke independently (no phantom edge across pen-lifts), normalises both sets to the unit box, and uses **symmetric Hausdorff** instead of Fréchet so the score is invariant to stroke order — appropriate for blank-canvas writing where the child can draw the strokes in any sequence.

How the implementation differs. Alt & Godau describe the continuous variant; this implementation uses Eiter & Mannila's discrete variant which is sufficient for digitised polylines. The flat-array vs 2D array choice is a cache-locality optimisation that doesn't change the algorithm's behaviour.

4.13 CoreML Letter Recognition

Description. A 40 × 40 grayscale CNN trained to classify a stroke image into one of 53 classes (A–Z, a–z, ß). Used after each freeWrite trial and inside the freeform writing mode.

Papers. (Underlying dataset and model design choices.) * Cohen, G., Afshar, S., Tapson, J., & van Schaik, A. (2017). EMNIST: an extension of MNIST to handwritten letters. International Joint Conference on Neural Networks (IJCNN), 2921–2926. * For Apple's on-device CNN execution: Apple Inc. (2023). Core ML Performance Best Practices. WWDC session materials.

Implementation. * `Core/LetterRecognizer.swift` — `CoreMLLetterRecognizer`. Marked `nonisolated` and runs every call inside `Task.detached(priority: .userInitiated)` so the inference path never executes on the main thread:

```swift return await Task.detached(priority: .userInitiated) { guard let model = Self.loadModelIfNeeded() else { return nil } guard let image = Self.renderToImage(points: points, canvasSize: canvasSize) else { return nil } let request = VNCoreMLRequest(model: model) request.imageCropAndScaleOption = .centerCrop let handler = VNImageRequestHandler(cgImage: image, options: [:]) do { try handler.perform([request]) let classifications = request.results as? [VNClassificationObservation] ?? [] return Self.makeResult(...) } catch { recognizerLogger.warning("Vision request failed: ...") return nil } }.value
```

- `renderToImage(points:canvasSize:)` rasterises the stroke into a 40 × 40 grayscale `CGContext`: black background, white stroke, centered with 4 px padding, line width 2.5 at model resolution. Y- axis is flipped so the top-left-origin touch points draw upright.
- `loadModelIfNeeded()` is guarded by `NSLock` so concurrent first calls don't double-load. Model is cached as a `nonisolated(unsafe)` static.
- If the `.mlpackage` cannot be found in any candidate bundle, the recognizer logs a warning and returns nil for every subsequent call — the UI shows a "KI-Modell nicht verfügbar" banner rather than silently failing.
- `ConfidenceCalibrator.calibrate(...)` post-processes the raw softmax probability with two adjustments:
- Confusable-pair penalty (15 % subtract for letters in `{C, c, O, o, S, s, V, v, W, w, X, x, Z, z, P, p, U, u, K, k}`).

- Optional history boost (10 % multiplicative bump if the child has ≥ 5 historical samples averaging ≥ 0.80 form score for the expected letter).

How the implementation differs. EMNIST is the conceptual model ancestor; the bundled `.mlpackage` is a custom-trained model on a German-handwriting-skewed dataset (the precise training corpus is the thesis author's). The confusable-pair list and history boost are the app's own calibration layer on top of the raw softmax.

4.14 A/B Testing Infrastructure

Description. Not a method per se; the app ships with the infrastructure to support thesis-grade A/B comparison between adaptive and non-adaptive conditions.

Implementation. * `Core/ThesisCondition.swift`:

```swift enum ThesisCondition: String, Codable, CaseIterable { case threePhase // full four-phase flow (legacy name preserved for Codable) case guidedOnly // guided phase only case control // guided phase, FixedAdaptationPolicy

```
    static func assign(participantId: UUID) -> ThesisCondition {
        let byte = participantId.uuid.0
        switch Int(byte) % ThesisCondition.allCases.count {
        case 0: return .threePhase
        case 1: return .guidedOnly
        default: return .control
        }
    }
```

} ```

- `ParticipantStore.participantId` — UUID generated on first call, persisted in UserDefaults so cohort assignment is stable across launches.
- `ParticipantStore.isEnrolled` — Settings toggle "Studienteilnahme". When false (default), `ThesisCondition.defaultForInstall` returns `.threePhase` so casual users never get a degraded condition.
- `LearningPhaseController.activePhases` returns `LearningPhase.allCases` for `.threePhase` and `[.guided]` for `.guidedOnly` / `.control`.
- `TracingDependencies.live` injects `FixedAdaptationPolicy(.standard)` for `.control` and `MovingAverageAdaptationPolicy()` for the others.
- Every dashboard write tags the active condition: `dashboardStore.recordPhaseSession(... condition: thesisCondition, ...)`.
- CSV / TSV / JSON export carries the condition as a column.

---
```

5. Font system

5.1 Primae font family

What it is. A custom-designed display font family bundled with the app. The repo ships **22 OTF files** under `Resources/Fonts/`: Primae-Regular, -Light, -Semilight, -Semibold, -Bold + cursive versions of each + a PrimaeText subfamily.

Why chosen. The Druckschrift (print) reference glyphs need to match a typography that's immediately readable to a German first- grader. Primae was chosen as a clean, sans-serif print face with balanced stroke widths and clearly distinguishable lowercase forms (unambiguous a vs g vs q).

How used. `Core/PrimaeLetterRenderer.swift` calls CoreText with `Primae-Regular` to rasterise a letter into a `UIImage` matching the canvas size. The bounding rect (`normalizedGlyphRect`) tells the checkpoint mapper where to place stroke checkpoints relative to the rendered glyph.

5.2 Playwrite AT (Schreibschrift)

What it is. Playwrite Österreich, an Austrian-school cursive font released by TypeTogether in 2023 under the **SIL Open Font License 1.1** (`Resources/Fonts/PlaywriteAT-OFL.txt`). Variable TTF.

Why chosen. The thesis originally targeted Österreichische Schulschrift 1995 (Pesendorfer's official font, mandated by the Austrian school authority RS 35/2022). That font's OTF was not available under a license compatible with bundling in a research app, and we could not source a single-letter version. Playwrite AT is a practising-Austrian-cursive that approximates Schulschrift 1995 well enough for the thesis claim of "supports a cursive script" while remaining legally redistributable.

How used. `Core/SchriftArt.swift` returns the font filename per case:

```
public var fontFileName: String {
    switch self {
    case .druckschrift:   return "Primae-Regular"
    case .schreibschrift: return "PlaywriteAT-Regular"
    case .grundschrift:   return "Grundschrift-Regular"            // not bundled
    case .vereinfachteAusgangschrift: return "VereinfachteAusgangschrift-Regular"  //
not bundled
    case .schulausgangsschrift:       return "Schulausgangsschrift-Regular"        //
not bundled
    }
}
```

The user-facing label for the cursive case is the generic "**Schreibschrift**", deliberately not "Schulschrift 1995", because the bundled font is not the official one.

5.3 SchriftArt switching

- `TracingDependencies` reads the persisted choice from `UserDefaults.standard.string(forKey: "de.flamingistan.buchstaben.selectedSchriftArt")` on init. Migration logic maps a legacy `"schulschrift1995"` value to `.schreibschrift`.
- `TracingViewModel.schriftArt` setter (line 30) clears the script stroke cache, the per-script font cache, re-renders the current letter image, and reloads stroke checkpoints in the new script's geometry.
- `SettingsView` shows only the two cases that are actually bundled (`druckschrift`, `schreibschrift`); the other three enum cases are scaffolded for future fonts.

5.4 PrimaeLetterRenderer

- `render(letter:size:schriftArt:)` returns a `UIImage` of the glyph drawn at the requested size using the SchriftArt's font.
- `normalizedGlyphRect(for:canvasSize:schriftArt:)` returns the bounding rectangle (in 0–1 normalized canvas space) of the glyph, used to map stroke checkpoints onto the rendered letter.
- `clearCache()` is called from `TracingViewModel.schriftArt.didSet` so a font switch doesn't serve stale rasterisations.

---

6. Data collection & export

6.1 ProgressStore (per letter)

File. `Core/ProgressStore.swift`. Stored at `Application Support/BuchstabenNative/progress.json` (atomic, debounced writes).

Schema.

```
struct LetterProgress: Codable, Equatable {
    var completionCount: Int = 0
    var bestAccuracy: Double = 0.0
    var lastCompletedAt: Date?
    var phaseScores: [String: Double]?              // "observe" / "direct" /
"guided" / "freeWrite"
    var speedTrend: [Double]?                        // last 5 sessions
    var paperTransferScore: Double?                  // 1.0 / 0.5 / 0.0
    var lastVariantUsed: String?                     // "variant" or nil
    var recognitionAccuracy: [Double]?               // legacy: last 10 confidences
    var recognitionSamples: [RecognitionSample]?     // last 10 full readings (pre-
dictedLetter + confidence + isCorrect)
    var freeformCompletionCount: Int?
}
```

`Store` wrapper additionally holds `completionDates: [Date]` to compute streaks.

6.2 ParentDashboardStore (sessions + phases)

File. `Core/ParentDashboardStore.swift`. Stored at `Application Support/BuchstabenNative/dashboard.json`.

Schema. * `LetterAccuracyStat(letter, accuracySamples)` — per letter, capped at 200 samples (rolling window). * `SessionDurationRecord(dateString, durationSeconds, condition)`. * `PhaseSessionRecord(letter, phase, completed, score, schedulerPriority, condition, formAccuracy?, tempoConsistency?, pressureControl?, rhythmScore?)` — the four optional fields are populated only on freeWrite rows.

Custom decoders default `condition` to `.threePhase` for pre-migration records.

6.3 StreakStore (engagement)

File. `Core/StreakStore.swift`. Stored at `Application Support/BuchstabenNative/streak.json`.

- `currentStreak`, `longestStreak`, `totalCompletions`, `completedLetters: Set<String>`.
- `RewardEvent` enum: `firstLetter`, `dailyGoalMet`, `streakDay3`, `streakWeek`, `streakMonth`, `allLettersComplete`, `perfectAccuracy`, `centuryClub`.
- `recordSession(date:lettersCompleted:accuracy:)` returns any newly earned `RewardEvent`s.

6.4 Research export

`Core/ParentDashboardExporter.swift` emits three formats. The CSV / TSV share a row builder so they stay in lock-step.

CSV / TSV header row (per-phase section):

```
letter,phase,completed,score,schedulerPriority,condition,
recognition_predicted,recognition_confidence,recognition_correct,
formAccuracy,tempoConsistency,pressureControl,rhythmScore
```

Column → thesis-analysis purpose: | Column | Purpose | |--------|---------| | `letter` | Per-letter slicing (alphabet effect, frequency analysis). | | `phase` | Which phase (observe / direct / guided / freeWrite) the session belonged to; needed for any GRR efficacy claim. | | `completed` | Selection criterion for "successful sessions". | | `score` | Phase-level accuracy / form score (0–1). | | `schedulerPriority` | Spaced-repetition scheduler's prediction at the time the letter was recommended; used as the IV in the scheduler-effectiveness Pearson r. | | `condition` | Thesis A/B arm — required for between-arm comparisons. | | `recognition_predicted` | Latest CoreML prediction for the letter (since recognition is per-letter, not per-phase, this is the most-recent reading at export time). | | `recognition_confidence` | Calibrated confidence 0–1 (post ConfidenceCalibrator). | | `recognition_correct` | Whether the prediction matched the expected letter. | | `formAccuracy` | Schreibmotorik Form dimension (Fréchet, weight 0.40). | | `tempoConsistency` | Schreibmotorik Tempo dimension (CV^2, weight 0.25). | | `pressureControl` | Schreibmotorik Druck dimension (force variance, weight 0.15). | | `rhythmScore` | Schreibmotorik Rhythmus dimension (active-time ratio, weight 0.20). |

Additional sections in the export: per-letter aggregates (sessionCount, averageAccuracy, trend slope), per-day session durations, phase-completion rates (one row per phase), aggregate freeWrite score, scheduler-effectiveness Pearson r, and the four average-Schreibmotorik-dimension rows when ≥ 1 freeWrite sample exists.

The first line of every file is `# participantId=<UUID>` so a researcher can align data across installs.

6.5 GDPR / DSGVO considerations

- All stores live in **Application Support** inside the app sandbox. Nothing is uploaded by default — `SyncCoordinator` is currently wired to `NullSyncService`.
- The participant UUID is **not derived from any device identifier**; it is randomly generated on first call (`UUID()` in `ParticipantStore.participantId`).
- Cohort enrolment is **opt-in** via a Settings toggle — non-enrolled installs are pinned to `.threePhase` and never contribute to A/B analysis.
- The CSV / TSV / JSON export is share-sheet driven: the parent / researcher exports manually; the app does no automatic transmission.
- No personal identifiers are stored: no name, no birthday, no email.
- CoreML inference is **on-device only** — stroke images never leave the iPad.
- For a full DSGVO Art. 13 disclosure note, the export's first line exposes the UUID so a participant can request deletion by reference.

7. Audio system

7.1 AudioEngine architecture (DOCUMENT, do NOT modify)

`Core/AudioEngine.swift` (382 lines). Implements `AudioControlling`.

- Built on `AVAudioEngine` + `AVAudioUnitTimePitch` for time-stretching playback (so audio speed tracks writing speed without pitch shift).
- Lifecycle observers (route change, interruption, suspend/resume) registered through a `nonisolated(unsafe) static` dictionary keyed by `ObjectIdentifier` so `deinit` can clean up without entering the MainActor.
- Public surface: `loadAudioFile(named:autoplay:)`, `setAdaptivePlayback(speed:horizontalBias:)`, `play()`, `stop()`, `restart()`, `suspendForLifecycle()`, `resumeAfterLifecycle()`, `cancelPendingLifecycleWork()`.

This file is annotated `STABLE & FRAGILE` in `CLAUDE.md` and `APP_REFERENCE.md`. Any change must be minimal and surgical (catch blocks, init/deinit structure are particularly load-bearing).

7.2 Audio triggering

- On letter load: `audio.loadAudioFile(named: firstAudio, autoplay: false)` queues the recorded letter pronunciation.
- On phase advance into observe: animation runs but audio stays idle (would loop silently behind onboarding otherwise).
- On `beginTouch`: `audio.loadAudioFile(named: ..., autoplay: false)` re-loads since `stop()` clears `currentFile`.
- On `updateTouch`: every frame computes `audio.setAdaptivePlayback(speed:, horizontalBias:)` where `speed = mapVelocityToSpeed(smoothedVelocity)` (slow tracing → 0.5, normal → 1.0, fast → 2.0) and `hBias = canvasNormalized.x * 2 - 1` for stereo panning.
- On `endTouch`: `audio.stop()` and the playback state machine forces idle.

7.3 Proximity-based playback (guided phase)

```
let shouldPlayForStroke = strokeTracker.isNearStroke
let shouldBeActive = shouldPlayForStroke
                     && smoothedVelocity >= playbackActivationVelocityThreshold
                     && feedbackIntensity > 0.3
playback.request(shouldBeActive ? .active : .idle, immediate: shouldBeActive)
```

`isNearStroke` is true when the touch is within `checkpointRadius * 3` of the next checkpoint. The 22 pt/s velocity threshold prevents audio on stationary touches.

7.4 Phase-dependent audio gating

`feedbackIntensity > 0.3` is the audio gate: * `.observe`: 1.0 → audio on (but touch is disabled anyway). * `.direct`: 1.0 → letter-name audio on first correct dot tap. * `.guided`: 0.6 → real-time proximity audio. * `.freeWrite`: 0.0 → audio silent (post-hoc TTS only).

---

8. Haptic system

8.1 Triggers

`HapticEngine.fire(_ event: HapticEvent)` is called from `TracingViewModel.updateTouch` and `tapDirectDot`. Five events:

Event	Trigger
`.strokeBegan`	First touch frame after `beginTouch`.
`.checkpointHit`	Tracker's `nextCheckpoint` increments, but the stroke isn't yet complete.
`.strokeCompleted`	A stroke's last checkpoint was just hit.
`.letterCompleted`	All strokes done.
`.offPath`	Wrong-dot tap in direct phase.

8.2 Phase-dependent intensity scaling

Every haptic call is gated on `feedbackIntensity > 0` (line 891, 918, 966 of `TracingViewModel`). Result: * `.observe` / `.direct`: full haptics (1.0). * `.guided`: full haptics (0.6 > 0; gate uses `> 0` not `> 0.6`, so haptics fire as long as feedback isn't fully off). * `.freeWrite`: zero haptics (0.0).

8.3 CoreHaptics patterns

`Core/HapticEngine.swift:118` — `CoreHapticsEngine.hapticPattern(for:)`:

Event	Pattern (intensity / sharpness / time)
`.strokeBegan`	one transient: 0.4 / 0.3 / 0 s
`.checkpointHit`	one transient: 0.6 / 0.7 / 0 s
`.strokeCompleted`	two transients: 0.8/0.5/0 s + 0.5/0.3/+0.1 s
`.letterCompleted`	three transients: 1.0/0.8/0 s + 0.8/0.5/+0.12 s + 0.6/0.3/+0.24 s (decay tail)
`.offPath`	one weak transient: 0.2 / 0.1 / 0 s

Falls back to `UIKitHapticEngine` (`UIImpactFeedbackGenerator` style mappings) when CoreHaptics isn't available, and `NullHapticEngine` for tests.

9. User interface

9.1 Navigation

```
MainAppView (root, requires onboarding complete)
  ├── WorldSwitcherRail (64 pt left rail; @AppStorage active world)
  │     ├── World 1: Schule                    (book.fill icon)
  │     ├── World 2: Werkstatt                 (pencil.tip)
  │     ├── World 3: Fortschritte / Sterne     (star.fill)
  │     └── Gear (2 s long-press → Parent area)
  └── World content
        ├── SchuleWorldView (guided 4-phase tracing)
        ├── WerkstattWorldView (freeform writing)
        └── FortschritteWorldView (star/streak/letter gallery)

Parent gate (fullScreenCover, 2 s long-press on gear):
ParentAreaView (NavigationSplitView)
  ├── Übersicht           → ParentDashboardView
  ├── Forschungs-Daten    → ResearchDashboardView
  ├── Einstellungen       → SettingsView
  └── Datenexport         → ExportCenterView (CSV / TSV / JSON share)
```

9.2 TracingCanvasView

- `Canvas` block draws (per cell):
- Whole-word image (word mode only) at full canvas size — so cursive ligatures connect across cells.
- Per-cell letter glyph via `PrimaeLetterRenderer.render`.
- Ghost lines from each stroke's checkpoints (gated on `showGhostForPhase || (showGhost && learningPhase != .freeWrite)`).
- Numbered start dots (gated on `phaseController.showCheckpoints`).
- Animation guide dot during observe.
- Active-stroke ink path (green, line width 8).
- Direction arrow during direct phase (1.2 s window).
- Two UIKit overlays:
- `PencilAwareCanvasOverlay` — captures pencil touches with pressure
  - azimuth.
- `TouchOverlay` — captures finger touches and a two-finger vertical swipe gesture (cycles audio variants).
- The KP overlay (`freeWriteKPOverlay()`) is rendered when `vm.overlayQueue.currentOverlay == .kpOverlay`.

9.3 Overlay queue

`Core/OverlayQueueManager.swift`. Five `CanvasOverlay` cases: `.frechetScore(CGFloat)`, `.kpOverlay`, `.recognitionBadge(RecognitionResult)`, `.paperTransfer(letter: String)`, `.celebration(stars: Int)`.

- Modal overlays (paperTransfer, celebration) carry **nil duration** → wait for explicit `.dismiss()`.
- Timed overlays (kpOverlay 3 s, recognitionBadge 3 s, frechetScore 1.5 s) auto-advance.
- `enqueueBeforeCelebration(_:)` slots a late-arriving recognition badge ahead of the celebration regardless of CoreML inference timing.

Canonical post-freeWrite order: kpOverlay → recognitionBadge → paperTransfer → celebration.

9.4 Settings

`Features/Dashboard/SettingsView.swift`: * **Schriftart**: Druckschrift / Schreibschrift (only the two bundled cases shown). * **Buchstabenreihenfolge**: motorSimilarity / wordBuilding / alphabetical. * **Freies Schreiben**: toggle for the freeform writing mode (default on). * **Forschung**: - “Schreiben auf Papier” — paper transfer toggle (default off). - “Studienteilnahme (A/B-Arm)” — opt-in cohort assignment toggle (default off). * **Hilfe**: “Einführung wiederholen” — resets onboarding state.

9.5 Parent dashboard

`Features/Dashboard/ParentDashboardView.swift`: * Übersicht: letters practised, current streak, longest streak, practice time (7 days). * Phasen: per-phase completion rates. * Übungsverlauf: 30-day practice trend chart. * Stärkste Buchstaben: top-5 by accuracy. * Übung nötig: letters below 0.6 accuracy. * Papier-Übertragung: per-letter paperTransferScore values. * Erkennungsgenauigkeit: per-letter rolling recognition averages. * DEBUG-only Forschungsmetriken section.

9.6 Research dashboard

`Features/Parent/ResearchDashboardView.swift`. Six sections: * **Participant header** — UUID + last-session condition. * **Schreibmotorik** — four metric tiles (Form / Tempo / Druck / Rhythmus) with weight annotations + last-session breakdown. * **KI-Erkennung** — per-letter latest prediction vs expected, with green/orange row tinting for correct/incorrect. * **Studienarm** — count of phase sessions per `ThesisCondition` arm. * **Spaced-Repetition-Effizienz** — Pearson r between scheduler priority and subsequent accuracy improvement. * **Phasen-Sessions (letzte 20)** — raw table of recent phase records. * **Buchstaben-Aggregate** — per-letter stats with trend slope.

9.7 Onboarding

`Features/Onboarding/OnboardingView.swift` (569 lines). Seven steps: welcome → traceDemo (observe phase animation of letter A) → directDemo (direct phase pulse + tap + arrow

simulation) → guidedDemo (finger emoji follows ink trail) → freeWriteDemo (green A) → rewardIntro (4 phase icons + star explanation) → complete.

Each step has a German-only Welcome / Anschauen / Richtung lernen / Nachspuren / Selbst schreiben / Sterne sammeln title and one or two sentences of body text. CTAs (“Los geht’s!”, “Weiter”, “Fertig!”) have explicit `.accessibilityLabel`. Adults can long-press the bottom “Für Eltern: gedrückt halten zum Überspringen” hint to skip.

---

10. Testing infrastructure

10.1 Test architecture

- **Swift Testing** (`@Test`, `@Suite`, `#expect`) for new tests; a small core of XCTest files retained from before the framework switch (per `LESSONS.md` instruction not to migrate existing XCTest).
- Tests live in `BuchstabenNativeTests/`. The package targets `.swiftLanguageMode(.v5)` for the test target only — the main target uses Swift 6 with `.defaultIsolation(MainActor.self)`. The v5 carve-out exists because `XCTestCase`'s inherited nonisolated initialiser conflicts with implicit MainActor isolation under Swift 6 strict checking.

10.2 Stubs (`TestFixtures.swift`)

- `StubAudio` — `AudioControlling` no-op.
- `StubHaptics` — `HapticEngineProviding` no-op.
- `StubProgressStore` — `ProgressStoring` returns empty data, ignores writes.
- `StubStreakStore`, `StubDashboardStore`, `StubOnboardingStore`, `StubNotificationCenter`, `NullLetterCache`, `StubResourceProvider` (with a synthetic A letter that has 50 checkpoints along y = 0.5).
- `NullSpeechSynthesizer` records every spoken phrase.
- `StubLetterRecognizer.alwaysReturn(predicted:confidence:)` for deterministic recognition.
- `MockAudio` (in `TracingViewModelTests.swift`) records every method call so tests can assert exact playback / lifecycle behaviour.

10.3 What's tested

(40+ test files, ~94 `@Test` declarations after the audit pass.)

- All four Schreibmotorik dimensions (`WritingAssessmentTests`).
- Letter scheduler weights, recency saturation, novelty tie-breaking (`LetterSchedulerTests`).
- `LetterOrderingStrategy` exact orderings (V3 spec compliance).
- Difficulty adaptation hysteresis (`DifficultyAdaptationTests`).
- Progress / dashboard / streak store JSON round-trip + corrupted-file recovery.
- CSV exporter columns including the new recognition-correct + Schreibmotorik dimensions.
- OverlayQueueManager ordering, dismissal, reset.
- Stroke tracker proximity, soundEnabled gating.
- Confidence calibrator confusable-pair penalty + history boost.
- End-to-end tracing session (`EndToEndTracingSessionTests`).
- TracingViewModel lifecycle (background / foreground churn, showGhost regression, retain cycles).

- Accessibility VoiceOver labels.
- Velocity → audio speed mapping monotonicity.
- `bucketStrokesByTargetLetter` word segmentation including overflow clamping and empty buckets.
- `RecognitionResult` Codable round-trip (RecognitionSample).
- `ChildSpeechLibrary` German prompt content + `NullSpeechSynthesizer` recording.

10.4 CI pipeline

`.github/workflows/ios-build.yml`: * Runs on **macos-26** with **Xcode 26.4**. * `xcodebuild test` against the iPad simulator scheme. * Uploads `/tmp/TestResults.xcresult` and `/tmp/xcode-test.log` as artifacts. * Greps the test log for `error:` / `#expect.*failed` / `XCTAssert.*failed` / `Test.*FAILED` and surfaces the matches.

`ipad-device-test.yml` runs on a self-hosted MacBook for physical- device validation; skipped on this run because the simulator job failed first.

10.5 Test coverage gaps (honest)

- **CoreMLLetterRecognizer**: no dedicated unit test for the model-loading + Vision-request path. The recognizer is exercised end-to-end via integration tests but the renderToImage / loadModel internals are unverified.
- **AudioEngine**: only sanity-skipped tests on simulator (lacks AVAudioSession route in CI).
- **DirectPhaseDotsOverlay tap routing**: the overlay's SwiftUI tap gestures are not exercised in unit tests; the underlying VM method `tapDirectDot(index:)` is.
- **PaperTransferView 3 s + 10 s timing**: the visual phase rotation uses real `Task.sleep` so it isn't deterministically tested. The state-mapping (.showLetter / .writePaper / .assess) is verified by inspection rather than assertion.
- **Variant strokes loading**: `LetterRepositoryTests` cover the bundle scan; the `toggleVariant()` method on the VM is unit-tested only via the no-variant case.
- **freeform recognition debounce**: the 1.2-second debounce is a real-time wait; unit tests exercise the controller's state but not the timing.

---

11. Thesis claims this code supports

#	Claim	Status	Evidence / gap
1	Four-step gradual release pedagogy for digital handwriting	**SUPPORTED**	`LearningPhase` has 4 cases in canonical order, `LearningPhaseController` enforces sequential advancement, each phase has distinct UI + scoring + scaffolding (see §3, §4.1).
2	Algorithmic spaced repetition applied to letter learning	**SUPPORTED**	`LetterScheduler` uses Ebbinghaus-style decay + accuracy + novelty weighting (see §4.10). `automatizationBonus` further deprioritises letters being automatised.
3	Fréchet distance as handwriting similarity metric	**SUPPORTED**	`FreeWriteScorer.discreteFrechetDistance` is the literal Eiter-Mannila implementation (see §4.12).
4	Four-dimension Schreibmotorik motor assessment on tablet	**SUPPORTED**	`WritingAssessment` struct holds all four dimensions with the documented 0.40/0.25/0.15/0.20 weights summing to 1.0. Each dimension has its own algorithm with a unit test. Dimensions persist on `PhaseSessionRecord` and export as separate CSV columns.
5	On-device letter recognition via CoreML CNN	**SUPPORTED**	`CoreMLLetterRecognizer` loads `GermanLetterRecognizer.mlpackage` and runs Vision inference on a `Task.detached`. The .mlpackage is bundled in `Resources/ML/` and includes `Manifest.json` + `Data/`. Inference never leaves the device.
6	Built-in A/B testing comparing adaptive vs. non-adaptive instruction	**SUPPORTED**	`ThesisCondition` provides three arms; `MovingAverageAdaptationPolicy` (adaptive) vs `FixedAdaptationPolicy` (control) selected by `TracingDependencies.live`; `ParticipantStore` persists stable UUID-derived assignment with explicit opt-in (see §4.14). Every dashboard write tags the condition so post-hoc analysis can split arms cleanly.
7	Fading feedback implements the guidance hypothesis	**SUPPORTED**	`feedbackIntensity` is a four-step fade (1.0 → 1.0 → 0.6 → 0.0), gates audio (> 0.3) and haptics (> 0) at three call sites in `updateTouch` (see §4.2).
8	Privacy-preserving on-device inference	**SUPPORTED**	The `CoreMLLetterRecognizer` runs entirely locally; `SyncCoordinator` is wired to `NullSyncService` so nothing is transmitted; the participant UUID is randomly generated, not derived from device identifiers; export is share-sheet driven (manual). The data flow is verifiable from `TracingDependencies.live` (see §6.5).

Partially supported claims worth flagging in the thesis:

- “Five-script support” — the SchriftArt enum has five cases but only Druckschrift (Primae) and Schreibschrift (Playwrite AT) are bundled. Grundschrift, Vereinfachte Ausgangsschrift, and Schulausgangsschrift are scaffolded for future work but won’t render without a font drop.
- “Full alphabet coverage” — 26 uppercase + 26 lowercase + Ä Ö Ü ß have stroke folders, but only 7 letters (A F I K L M O) ship full audio + calibrated checkpoints. The other letters render as glyphs but the empty-strokes path skips observe and direct.

---

12. Bibliography

In APA 7 format, sorted alphabetically.

Alamargot, D., & Morin, M.-F. (2015). Does handwriting on a tablet screen affect students' graphomotor execution? A comparison between Grades Two and Nine. Human Movement Science, 44, 32–41. https://doi.org/10.1016/j.humov.2015.08.011

Alt, H., & Godau, M. (1995). Computing the Fréchet distance between two polygonal curves. International Journal of Computational Geometry & Applications, 5(1n02), 75–91. https://doi.org/10.1142/S0218195995000064

Berninger, V. W., Abbott, R. D., Jones, J., Wolf, B. J., Gould, L., Anderson-Youngstrom, M., Shimada, S., & Apel, K. (2006). Early development of language by hand: Composing, reading, listening, and speaking connections; three letter-writing modes; and fast mapping in spelling. Developmental Neuropsychology, 29(1), 61–92. https://doi.org/10.1207/s15326942dn2901_5

Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., & Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. Psychological Bulletin, 132(3), 354–380. https://doi.org/10.1037/0033-2909.132.3.354

Cohen, G., Afshar, S., Tapson, J., & van Schaik, A. (2017). EMNIST: An extension of MNIST to handwritten letters. In Proceedings of the International Joint Conference on Neural Networks (IJCNN) (pp. 2921–2926). IEEE. https://doi.org/10.1109/IJCNN.2017.7966217

Danna, J., & Velay, J.-L. (2015). Basic and supplementary sensory feedback in handwriting. Frontiers in Psychology, 6, 169. https://doi.org/10.3389/fpsyg.2015.00169

Ebbinghaus, H. (1885). Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie. Duncker & Humblot.

Eiter, T., & Mannila, H. (1994). Computing discrete Fréchet distance (Tech. Rep. No. CD-TR 94/64). Christian Doppler Laboratory for Expert Systems, Technische Universität Wien.

Fisher, D., & Frey, N. (2013). Better learning through structured teaching: A framework for the gradual release of responsibility (2nd ed.). ASCD.

Fréchet, M. (1906). Sur quelques points du calcul fonctionnel. Rendiconti del Circolo Matematico di Palermo, 22(1), 1–72. https://doi.org/10.1007/BF03018603

Grundschulverband. (2024). Grundschrift — Förderkartei zur Schreibmotorik. Frankfurt am Main: Grundschulverband.

Kultusministerkonferenz. (2024). Vereinbarung zur Arbeit in der Grundschule (revised). Berlin: Kultusministerkonferenz.

Marquardt, C., & Söhl, K. (2016). Schreibmotorik: Schreiben lernen leicht gemacht — Eine Praxishilfe der STEP-Schreibmotorik-Initiative. Cornelsen.

Pearson, P. D., & Gallagher, M. C. (1983). The instruction of reading comprehension. Contemporary Educational Psychology, 8(3), 317–344. https://doi.org/10.1016/0361-476X(83)90019-X

Schmidt, R. A., & Lee, T. D. (2005). Motor control and learning: A behavioral emphasis (4th ed.). Human Kinetics.

Thibon, L. S., Gerber, S., & Kandel, S. (2018). The elaboration of motor programs for the automation of letter production. Acta Psychologica, 182, 200–211. https://doi.org/10.1016/j.actpsy.2017.11.014

Vygotsky, L. S. (1978). Mind in society: The development of higher psychological processes (M. Cole, V. John-Steiner, S. Scribner, & E. Souberman, Eds.). Harvard University Press.

End of document. Last updated against the source tree at HEAD on the date this file was last committed.